

cero o un uno en la señal GPIO 9, para apagar y encender el primer LED de la placa auxiliar respectivamente (un cero apaga y un uno enciende el LED).

Para ello tenemos los puertos GPSETn y GPCLRn, donde GPSETn pone un 1 y GPCLRn pone un 0. En principio parece enrevesado el tener que usar dos puertos distintos para escribir en el puerto GPIO, pero no olvidemos que para ahorrar recursos varios pines están empaquetados en una palabra de 32 bits. Si sólo tuviéramos un puerto y quisiéramos alterar un único pin tendríamos que leer el puerto, modificar el bit en cuestión sin tocar los demás y escribir el resultado de nuevo en el puerto. Por suerte esto no es necesario con puertos separados para setear y resetear, tan sólo necesitamos una escritura en puerto poniendo a 1 los bits que queramos setear/resetear y a 0 los bits que no queramos modificar.

En la figura 4.4 vemos cómo está hecho el conexionado de la placa auxiliar.

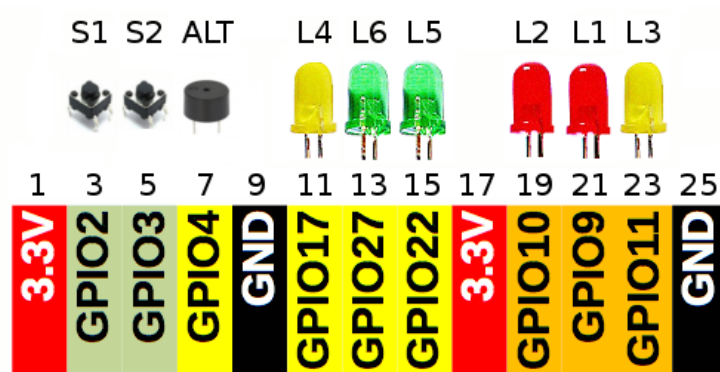


Figura 4.4: Correspondencia LEDs y GPIO

En nuestro primer ejemplo de Bare Metal sólo vamos a encender el primer LED rojo de la placa auxiliar, que como hemos dicho se corresponde con el GPIO 9 así que tendremos que actuar sobre el bit 9 del registro GPSET0.

Resumiendo, los puertos a los que accedemos para encender y apagar el LED vienen indicados en la figura 4.5.

El siguiente código (listado 4.2) muestra cómo hemos de proceder.

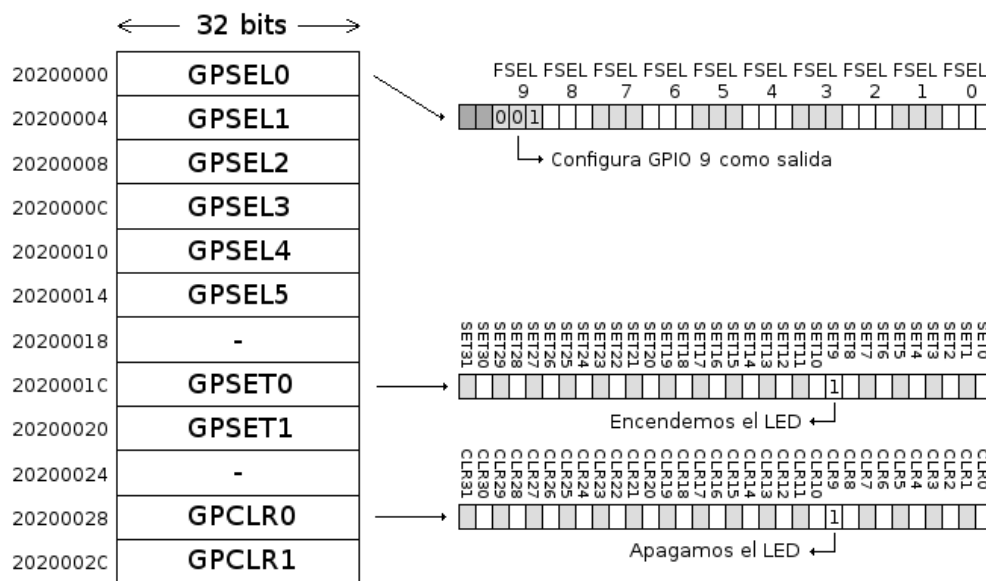


Figura 4.5: Puertos LED

Listado 4.2: esbn2.s

```

.set GPBASE, 0x20200000
.set GPFSEL0, 0x00
.set GPSET0, 0x1c

.text
    ldr    r0, =GPBASE
/* guia bits          xx999888777666555444333222111000*/
    mov    r1, #0b00001000000000000000000000000000
    str    r1, [r0, #GPFSEL0] @ Configura GPIO 9
/* guia bits          10987654321098765432109876543210*/
    mov    r1, #0b00000000000000000000000000000000
    str    r1, [r0, #GPSET0] @ Enciende GPIO 9
infi:    b    infi

```

El acceso a los puertos lo hemos hecho usando la dirección base donde están mapeados los periféricos 0x20200000. Cargamos esta dirección base en el registro r0 y codificamos los accesos a los puertos E/S con direccionamiento a memoria empleando distintas constantes como desplazamiento en función del puerto al que queramos acceder.

El código simplemente escribe dos constantes en dos puertos: GPFSEL0 y GPSET0. Con la primera escritura configuramos el LED como salida y con la segunda escritura lo encendemos, para finalmente entrar en un bucle infinito con `infi: b infi`.

Otros puertos

Ya hemos explicado los puertos que vamos a usar en este capítulo, pero el dispositivo GPIO tiene más puertos.

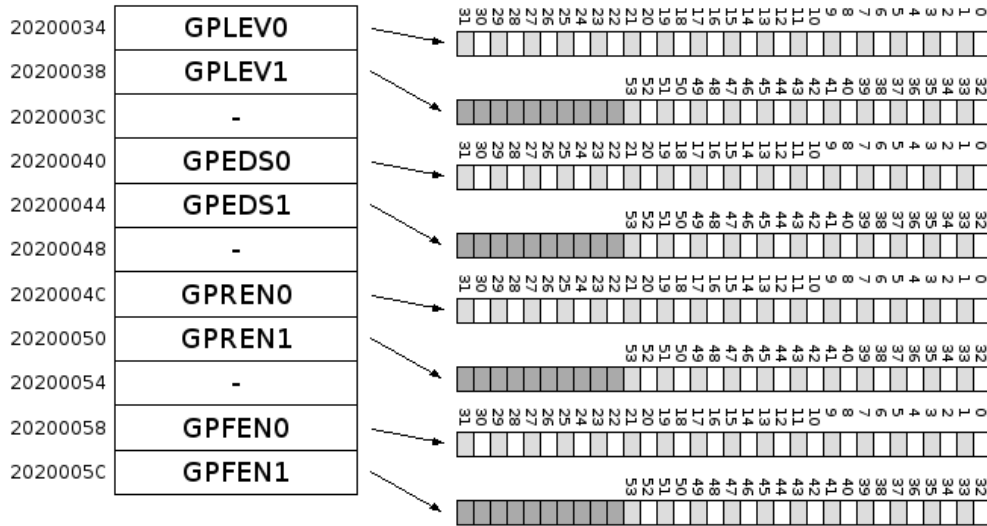


Figura 4.6: Otros puertos del GPIO (1ª parte)

En la figura 4.6 tenemos los siguientes:

- **GPLEVn**. Estos puertos devuelven el valor del pin respectivo. Si dicho pin está en torno a 0V devolverá un cero, si está en torno a 3.3V devolverá un 1.
- **GPEDSn**. Sirven para detectar qué pin ha provocado una interrupción en caso de usarlo como lectura. Al escribir en ellos también podemos notificar que ya hemos procesado la interrupción y que por tanto estamos listos para que nos vuelvan a interrumpir sobre los pines que indiquemos.
- **GPRENn**. Con estos puertos enmascaramos los pines que queremos que provoquen una interrupción en flanco de subida, esto es cuando hay una transición de 0 a 1 en el pin de entrada.
- **GPFENn**. Lo mismo que el anterior pero en flanco de bajada.

El resto de puertos GPIO se muestran en la figura 4.7.

Estos registros son los siguientes:

- **GPHENn**. Enmascaramos los pines que provocarán una interrupción al detectar un nivel alto (3.3V) por dicho pin.

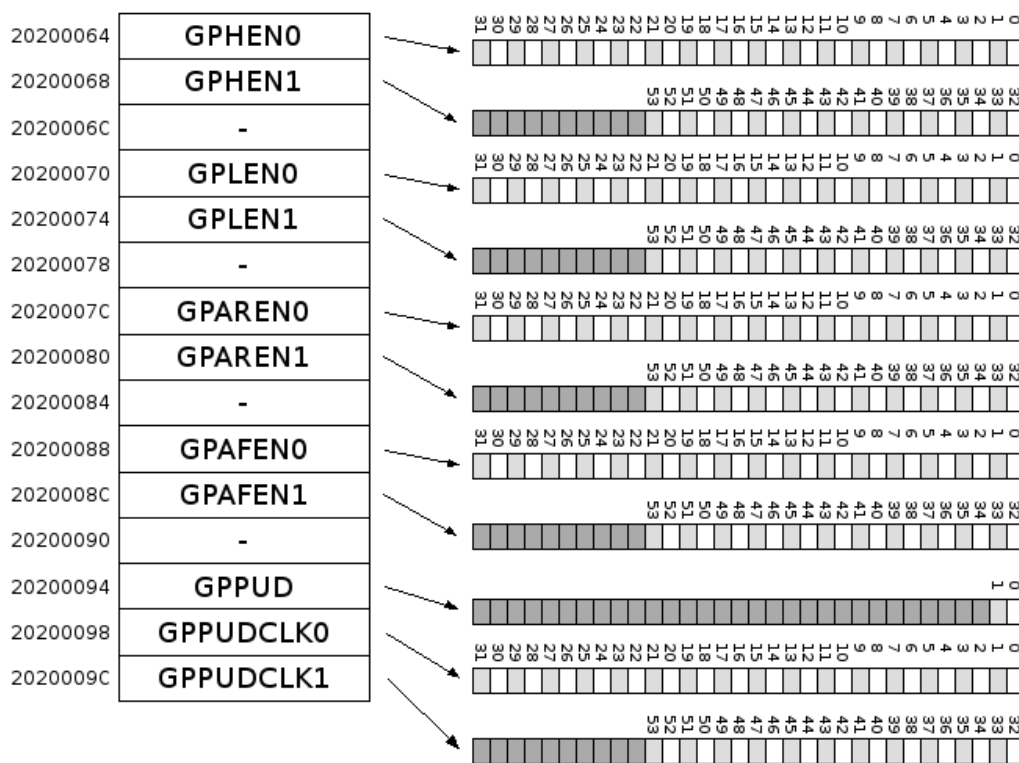


Figura 4.7: Otros puertos del GPIO (2ª parte)

- **GPLENn**. Lo mismo que el anterior pero para un nivel bajo (0V).
- **GPARENn** y **GPAFENn**. Tienen funciones idénticas a **GPRENn** y **GPFENn**, pero permiten detectar flancos en pulsos de poca duración.
- **GPPUD** y **GPPUDCLKn**. Conectan resistencias de pull-up y de pull-down sobre los pines que deseemos. Para más información ver el último ejemplo del siguiente capítulo.

4.2.2. Temporizador del sistema

El temporizador del sistema es un reloj que funciona a 1MHz y en cada paso incrementa un contador de 64bits. Este contador viene muy bien para implementar retardos o esperas porque cada paso del contador se corresponde con un microsegundo. Los puertos asociados al temporizador son los de la figura 4.8. Básicamente encontramos un contador de 64 bits y cuatro comparadores. El contador está dividido en dos partes, la parte baja **CLO** y la parte alta **CHI**. La parte alta no nos resulta

interesante, porque tarda poco más de una hora ($2^{32} \mu s$) en incrementarse y no va asociado a ningún comparador.

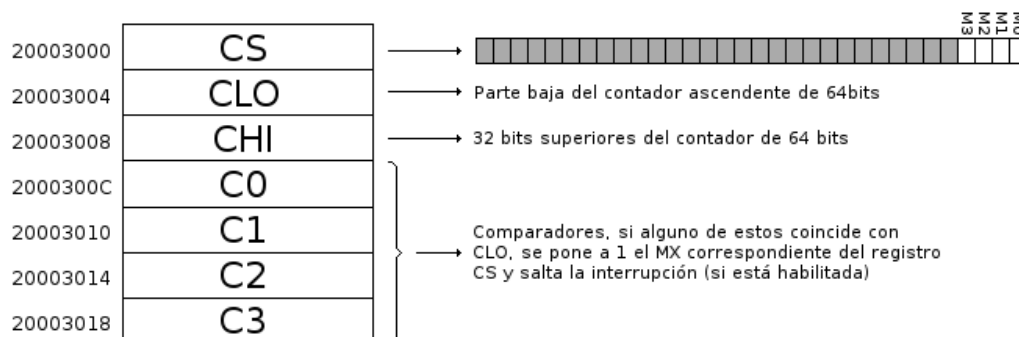


Figura 4.8: System Timer

Los comparadores son puertos que se pueden modificar y se comparan con CLO. En el momento que uno de los 4 comparadores coincida y estén habilitadas las interrupciones para dicho comparador, se produce una interrupción y se activa el correspondiente bit Mx asociado al puerto CS (para que en la rutina de tratamiento de interrupción o RTI sepamos qué comparador ha provocado la interrupción). Los comparadores C0 y C2 los emplea la GPU internamente, por lo que nosotros nos ceñiremos a los comparadores C1 y C3.

Las interrupciones las veremos en la siguiente lección. Por ahora sólo vamos a acceder al puerto CLO para hacer parpadear un LED a una frecuencia determinada. El esquema funcional del System Timer se muestra en la figura 4.9.

4.3. Ejemplos de programas Bare Metal

4.3.1. LED parpadeante con bucle de retardo

La teoría sobre encender y apagar el LED la sabemos. Lo más sencillo que podemos hacer ahora es hacer que el LED parpadee continuamente. Vamos a introducir el siguiente programa en la Raspberry, antes de probarlo piensa un poco cómo se comportaría el código del listado 4.3.

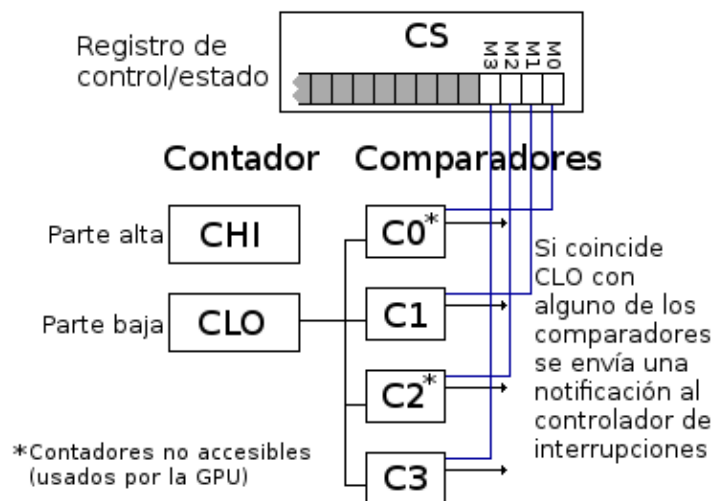


Figura 4.9: Esquema funcional del System Timer

Listado 4.3: esbn3.s

```

.set    GPBASE,    0x20200000
.set    GPFSEL0,   0x00
.set    GPSET0,    0x1c
.set    GPCLR0,    0x28

.text
    ldr    r0, =GPBASE
/* guia bits          xx999888777666555444333222111000*/
    mov    r1, #0b00001000000000000000000000000000
    str    r1, [r0, #GPFSEL0] @ Configura como salida
/* guia bits          10987654321098765432109876543210*/
bucle:  mov    r1, #0b0000000000000000000000001000000000
    str    r1, [r0, #GPSET0] @ Enciende
    mov    r1, #0b0000000000000000000000001000000000
    str    r1, [r0, #GPCLR0] @ Apaga
    b      bucle

```

Para compilar y ejecutar este ejemplo sigue los pasos descritos en 4.1.2. Al ejecutar el kernel.ing resultante comprobamos que el LED no parpadea sino que está encendido con menos brillo del normal. En realidad sí que lo hace, sólo que nuestro ojo es demasiado lento como para percibirlo. Lo siguiente será ajustar la cadencia del parpadeo a un segundo para que podamos observar el parpadeo. La secuencia sería apagar el LED, esperar medio segundo, encender el LED, esperar otro medio segundo y repetir el bucle. Sabemos que el procesador de la Raspberry corre a 700MHz por lo que vamos a suponer que tarde un ciclo de este reloj en ejecutar cada

4.3.2. LED parpadeante con temporizador

Viendo lo poco preciso que es el temporizar con el bucle de retardo, vamos a sincronizar leyendo continuamente el valor del `System Timer`. Como el temporizador va a 1MHz, para temporizar medio segundo lo único que tenemos que hacer es esperar a que el contador se incremente en medio millón. El código final quedaría así:

Listado 4.5: esbn5.s

```

.set GPBASE, 0x20200000
.set GPFSEL0, 0x00
.set GPSET0, 0x1c
.set GPCLR0, 0x28
.set STBASE, 0x20003000
.set STCLR0, 0x04

.text
    ldr    r0, =GPBASE
/* guia bits          xx999888777666555444333222111000*/
    mov    r1, #0b00001000000000000000000000000000
    str    r1, [r0, #GPFSEL0] @ Configura GPIO 9
/* guia bits          10987654321098765432109876543210*/
    mov    r1, #0b0000000000000000000000001000000000
    ldr    r2, =STBASE

bucle: bl    espera          @ Salta a rutina de espera
        str    r1, [r0, #GPSET0]
        bl    espera          @ Salta a rutina de espera
        str    r1, [r0, #GPCLR0]
        b     bucle

/* rutina que espera medio segundo */
espera: ldr    r3, [r2, #STCLR0] @ Lee contador en r3
        ldr    r4, =500000
        add    r4, r3          @ r4= r3+medio millón
ret1:   ldr    r3, [r2, #STCLR0]
        cmp    r3, r4          @ Leemos CLR hasta alcanzar
        bne    ret1           @ el valor de r4
        bx     lr

```

4.3.3. Sonido con temporizador

Este ejemplo es exactamente el mismo que el anterior, tan sólo hemos cambiado el pin del LED (GPIO 9) por el pin asociado al altavoz de nuestra placa de expan-

sión (GPIO 4). También modificamos el tiempo de espera para producir un sonido audible.

Vamos a producir un tono de 440 Hz. Para ello generamos una onda cuadrada por dicho pin, que no es más que una serie de ceros y unos consecutivos de idéntica duración. A esta duración la llamamos semi-periodo, y es la que queremos calcular. Como el periodo es el inverso de la frecuencia, tenemos que $periodo = 1/(440s^{-1}) = 2,272 \times 10^{-3}s$, por lo que el semi-periodo buscado es $2,272 \times 10^{-3}s/2 = 1,136 \times 10^{-3}s$ o lo que es lo mismo, 1136 microsegundos.

Listado 4.6: Parte de esbn6.s

```

        ldr    r0, =GPBASE
/* guia bits          xx999888777666555444333222111000*/
        mov    r1, #0b00000000000000000000100000000000
        str    r1, [r0, #GPFSEL0] @ Configura GPIO 4
/* guia bits          10987654321098765432109876543210*/
        mov    r1, #0b000000000000000000000000000010000
        ldr    r2, =STBASE

bucle:  bl     espera          @ Salta a rutina de espera
        str    r1, [r0, #GPSET0]
        bl     espera          @ Salta a rutina de espera
        str    r1, [r0, #GPCLR0]
        b      bucle

/* rutina que espera 1136 microsegundos */
espera: ldr    r3, [r2, #STCL0] @ Lee contador en r3
        ldr    r4, =1136
        add    r4, r3          @ r4= r3 + 1136
ret1:   ldr    r3, [r2, #STCL0]
        cmp    r3, r4          @ Leemos CL0 hasta alcanzar
        bne    ret1           @ el valor de r4
        bx     lr

```

Capítulo 5

Interrupciones hardware

Contenido

5.1	Lectura previa	104
5.1.1	El sistema de interrupciones del ARM	104
5.1.2	Rutina de tratamiento de interrupción	109
5.1.3	Pasos para configurar las interrupciones	110
5.1.4	El controlador de interrupciones	112
5.1.5	Ejemplo. Encender LED rojo a los 4 segundos	114
5.1.6	Ejemplos de aplicación	118
5.1.7	Parpadeo de todos los LEDs	119
5.1.8	Control de LEDs rojos con pulsadores	123
5.1.9	Parpadeo secuencial de LEDs con sonido por altavoz	127
5.1.10	Manejo de FIQs y sonidos distintos para cada LED	133
5.1.11	Control de luces/sonido con pulsadores en lugar temporizadores	138
5.2	Ejercicios	142
5.2.1	Todo con IRQs	142
5.2.2	Alargar secuencia a 10 y parpadeo	142
5.2.3	Tope de secuencia y limitar sonido	142
5.2.4	Reproductor de melodía sencilla	143

Objetivos: En esta sesión vamos a realizar programas que utilizan dispositivos de E/S haciendo uso del sistema de interrupciones hardware. Para poder programar los distintos parámetros que configuran el entorno de las interrupciones es necesario

Basic.

El puerto INTFIQCON se programa con los 8 bits inferiores, indicando en el bit 7 si queremos habilitar la fuente, y en los bits del 0 al 6 ponemos el índice de la fuente que se corresponde con la lista. A diferencia de las IRQ, con las FIQ sólo podemos atender a una fuente de interrupción.

5.1.5. Ejemplo. Encender LED rojo a los 4 segundos

Se trata de programar el comparador y las interrupciones para que transcurrido un tiempo determinado se produzca una interrupción, dentro de la cual se encienda el LED. Es un caso muy sencillo porque sólo se va a producir una interrupción que viene de una sola fuente, por lo que en la RTI lo único que haremos es encender el LED.

El diagrama que vamos a usar es el siguiente.

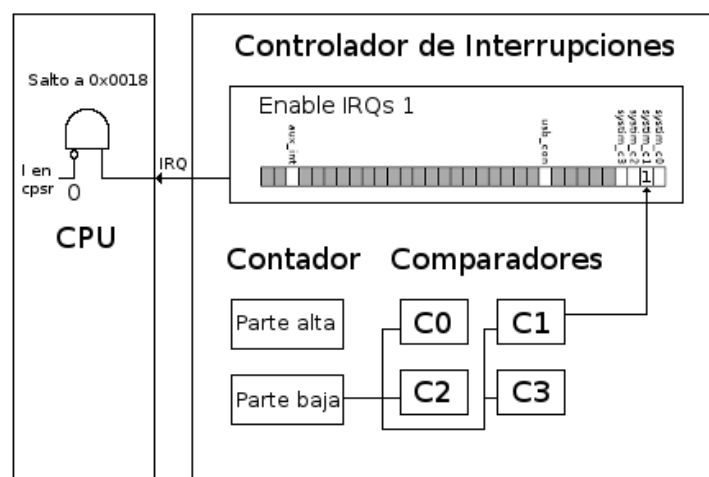


Figura 5.7: Interrupciones

1. Escribimos en el vector de interrupciones

Invocamos la macro para una IRQ, pasándole la etiqueta de nuestra RTI `irq_handler`.

```
ADDEXC 0x18, irq_handler
```

2. Inicializamos punteros de pila

La única forma de acceder a los registros `sp_irq` y `sp_fiq` es cambiando de modo y modificando el registro `sp` correspondiente.

El modo viene indicado en la parte más baja del registro `cpsr`, el cual modificaremos con la instrucción especial `msr`. En la figura 5.1 vemos el contenido completo del registro `cpsr`. Como `cpsr` es un registro muy heterogéneo, usamos sufijos para acceder a partes concretas de él. En nuestro caso sólo nos interesa cambiar el byte bajo del registro, añadimos el sufijo `_c` llamándolo `cpsr_c`, para no alterar el resto del registro. Esta parte comprende el modo de operación y las máscaras globales de las interrupciones. Otra referencia útil es `cpsr_f` que modifica únicamente la parte de flags (byte alto). Las otras 3 referencias restantes apenas se usan y son `cpsr_s` (Status) para el tercer byte, `cpsr_x` (eXtended) para el segundo byte y `cpsr_csxf` para modificar los 4 bytes a la vez.

En la siguiente tabla vemos cómo se codifica el modo de operación.

Hex	Binario	Modo de operación
0x10	10000	Usuario
0x11	10001	FIQ
0x12	10010	IRQ
0x13	10011	Supervisor
0x16	10110	Monitor seguro
0x17	10111	Abort
0x1B	11011	Indefinido
0x1F	11111	Sistema

Como las interrupciones globales de IRQ y FIQ están desactivadas (estado por defecto tras el reset), mantenemos a 1 dichos bits.

El código que inicializa los punteros de pila es el siguiente:

```

mov    r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
msr    cpsr_c, r0
mov    sp, #0x8000
mov    r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
msr    cpsr_c, r0
mov    sp, #0x8000000
```

En concreto a 0x8000 y 0x8000000 para los modos IRQ y Supervisor respectivamente.

3. Código de inicialización ajeno a interrupciones

En el ejemplo que tenemos entre manos se trata de configurar los puertos GPIO de entrada y de salida, inicializar temporizadores. En casos más complejos tendríamos que inicializar estructuras de datos, rellenar las tablas que sean precalculadas y en general cualquier tarea de inicialización requerida para hacer funcionar nuestro programa.

El código para asignar el sentido al pin GPIO 9 es el siguiente:

```

        ldr    r0, =GPBASE
/* guia bits          xx999888777666555444333222111000*/
        mov    r1, #0b00001000000000000000000000000000
        str    r1, [r0, #GPFSEL0]

```

Luego programamos el comparador para que salte la interrupción a los 4,19 segundos:

```

        ldr    r0, =STBASE
        ldr    r1, [r0, #STCL0]
        add    r1, #0x400000    @4,19 segundos
        str    r1, [r0, #STC1]

```

4. Inicializamos interrupciones localmente

Consiste en escribir en los puertos asociados dependiendo de las fuentes que querramos activar. En este primer ejemplo habilitamos el comparador C1 del temporizador como fuente de interrupción:

```

        ldr    r0, =INTBASE
        mov    r1, #0b0010
        str    r1, [r0, #INTENIRQ1]

```

5. Habilitamos interrupciones globalmente

Se trata de poner a cero el bit correspondiente en `cpsr`. El siguiente código habilita interrupciones del tipo IRQ:

```

        mov    r0, #0b01010011    @ Modo SVC, IRQ activo
        msr    cpsr_c, r0

```

6. Resto del programa principal

Como hemos adelantado, en todos nuestros ejemplos será un bucle infinito:

```

bucle:  b      bucle

```

A continuación mostramos el listado del ejemplo completo:

Listado 5.1: inter1.s

```

        .include "inter.inc"
.text
/* Agrego vector interrupción */
        ADDEXC    0x18, irq_handler

/* Inicializo la pila en modos IRQ y SVC */
        mov      r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
        msr      cpsr_c, r0
        mov      sp, #0x8000
        mov      r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
        msr      cpsr_c, r0
        mov      sp, #0x8000000

/* Configuro GPIO 9 como salida */
        ldr      r0, =GPBASE
/* guia bits          xx999888777666555444333222111000*/
        mov      r1, #0b00001000000000000000000000000000
        str      r1, [r0, #GPFSEL0]

/* Programo contador C1 para futura interrupción */
        ldr      r0, =STBASE
        ldr      r1, [r0, #STCLO]
        add      r1, #0x400000        @4,19 segundos
        str      r1, [r0, #STC1]

/* Habilito interrupciones, local y globalmente */
        ldr      r0, =INTBASE
        mov      r1, #0b0010
        str      r1, [r0, #INTENIRQ1]
        mov      r0, #0b01010011    @ Modo SVC, IRQ activo
        msr      cpsr_c, r0

/* Repetir para siempre */
bucle:  b        bucle

/* Rutina de tratamiento de interrupción */
irq_handler:
        push     {r0, r1}            @ Salvo registros

        ldr      r0, =GPBASE
/* guia bits          10987654321098765432109876543210*/
        mov      r1, #0b0000000000000000000000001000000000

```

```

    str    r1, [r0, #GPSET0] @ Enciendo LED

    pop    {r0, r1}          @ Recupero registros
    subs   pc, lr, #4        @ Salgo de la RTI

```

Observamos que la RTI es muy sencilla, aparte del esqueleto tenemos tres instrucciones encargadas de encender el LED en cuestión.

5.1.6. Ejemplos de aplicación

Vamos a crear un archivo `inter.inc` donde guardaremos las constantes asociadas a los puertos y también la macro `ADDEXC`, esta última se explica en detalle en el apéndice A. De esta forma evitamos escribir siempre las mismas constantes, haciendo el código más sencillo de mantener.

Listado 5.2: `inter.inc`

```

.macro    ADDEXC    vector, dirRTI
    ldr    r1, =(\dirRTI-\vector+0xa7fffffb)
    ROR    r1, #2
    str    r1, [r0, #\vector]
.endm

.set      GPBASE,      0x20200000
.set      GPFSEL0,     0x00
.set      GPFSEL1,     0x04
.set      GPFSEL2,     0x08
.set      GPSET0,      0x1c
.set      GPCLR0,      0x28
.set      GPEDS0,      0x40
.set      GPFEN0,      0x58
.set      GPPUD,       0x94
.set      GPPUDCLK0,   0x98
.set      STBASE,      0x20003000
.set      STCS,        0x00
.set      STCLO,       0x04
.set      STC1,        0x10
.set      STC3,        0x18
.set      INTBASE,     0x2000b000
.set      INTFIQCON,   0x20c
.set      INTENIRQ1,   0x210
.set      INTENIRQ2,   0x214

```

El método para incluir el código fuente de un fichero dentro de otro es mediante la macro `.include`, todos nuestros ficheros comenzarán con lo siguiente.

```
.include "inter.inc"
```

5.1.7. Parpadeo de todos los LEDs

Sería hacer lo mismo que en la lección anterior pero empleando interrupciones y aplicando la salida simultáneamente a los 6 LEDs en lugar de sólo al primero. La novedad en lo que a interrupciones se refiere consiste en reprogramar el comparador C1 cada vez que se produzca una interrupción, de esta forma conseguimos interrupciones periódicas en lugar de una única interrupción.

Veamos el código:

Listado 5.3: inter2.s

```
.include "inter.inc"

.text
/* Agrego vector interrupción */
ADDEXC 0x18, irq_handler

/* Inicializo la pila en modos IRQ y SVC */
mov     r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
msr     cpsr_c, r0
mov     sp, #0x8000
mov     r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
msr     cpsr_c, r0
mov     sp, #0x8000000

/* Configuro GPIOs 9, 10, 11, 17, 22 y 27 como salida */
ldr     r0, =GPBASE
mov     r1, #0b00001000000000000000000000000000
str     r1, [r0, #GPFSEL0]
/* guia bits          xx999888777666555444333222111000*/
ldr     r1, =0b00000000001000000000000000000001001
str     r1, [r0, #GPFSEL1]
ldr     r1, =0b00000000001000000000000000001000000
str     r1, [r0, #GPFSEL2]

/* Programo contador C1 para dentro de 2 microsegundos */
ldr     r0, =STBASE
ldr     r1, [r0, #STCLO]
add     r1, #2
str     r1, [r0, #STC1]
```



```

/* Habilito interrupciones, local y globalmente */
    ldr    r0, =INTBASE
    mov    r1, #0b0010
    str    r1, [r0, #INTENIRQ1]
    mov    r0, #0b01010011    @ Modo SVC, IRQ activo
    msr    cpsr_c, r0

/* Repetir para siempre */
bucle:  b    bucle

/* Rutina de tratamiento de interrupción */
irq_handler:
    push   {r0, r1, r2}

/* Conmuto variable de estado del LED */
    ldr    r0, =ledst    @ Leo puntero a v. ledst
    ldr    r1, [r0]       @ Leo variable
    eors   r1, #1         @ Invierto bit 0, act. flag Z
    str    r1, [r0]       @ Escribo variable

/* Enciendo o apago todos los LEDs en función del flag Z */
    ldr    r0, =GPBASE
/* guia bits      10987654321098765432109876543210*/
    ldr    r1, =0b00001000010000100000111000000000
    streq   r1, [r0, #GPSET0]
    strne   r1, [r0, #GPCLR0]

/* Reseteo estado interrupción de C1 */
    ldr    r0, =STBASE
    mov    r1, #0b0010
    str    r1, [r0, #STCS]

/* Programo siguiente interrupción medio segundo después */
    ldr    r1, [r0, #STCLO]
    ldr    r2, =500000    @1 Hz
    add    r1, r2
    str    r1, [r0, #STC1]

/* Recupero registros y salgo */
    pop    {r0, r1, r2}
    subs   pc, lr, #4

```

```
/* Ubicación de la variable ledst */
ledst: .word 0
```

Y vamos enumerando, por orden, los pasos que hemos seguido. En primer lugar apuntamos a nuestra RTI en el vector de interrupciones:

```
ADDEXC 0x18, irq_handler
```

Luego inicializamos los punteros de pila:

```
mov     r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
msr     cpsr_c, r0
mov     sp, #0x8000
mov     r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
msr     cpsr_c, r0
mov     sp, #0x8000000
```

Lo siguiente es configurar los pines GPIO asociados a los 6 LEDs como salidas:

```
ldr     r0, =GPBASE
mov     r1, #0b00001000000000000000000000000000
str     r1, [r0, #GPFSEL0]
/* guia bits          xx999888777666555444333222111000*/
ldr     r1, =0b0000000000100000000000000000001001
str     r1, [r0, #GPFSEL1]
ldr     r1, =0b00000000001000000000000000001000000
str     r1, [r0, #GPFSEL2]
```

Preparamos el comparador C1 para que al cabo de dos microsegundos nos proporcione la primera interrupción:

```
ldr     r0, =STBASE
ldr     r1, [r0, #STCLO]
add     r1, #2
str     r1, [r0, #STC1]
```

Para después habilitar las interrupciones asociadas al comparador C1:

```
ldr     r0, =INTBASE
mov     r1, #0b0010
str     r1, [r0, #INTENIRQ1]
```

Y finalmente habilitar las interrupciones IRQ globalmente, entrando luego en el bucle infinito:

```
mov     r0, #0b01010011    @modo SVC, IRQ activo
msr     cpsr_c, r0
bucle:  b     bucle
```

Ya hemos terminado con el programa principal, que como veremos más adelante va a ser siempre muy parecido.

Lo interesante está en la RTI, que es donde hacemos parpadear los LEDs y configuramos el comparador para la siguiente interrupción.

El estado de los LEDs (si están apagados o encendidos) lo guardamos en la variable `ledst`, que conmutamos entre cero y uno mediante un `OR` exclusivo. Al actualizar los `flags` tras esta operación, tenemos que si el resultado fue cero nos lo indica el `flag Z` activo, mientras que estará inactivo en el caso contrario (resultado 1). Mediante las instrucciones de ejecución condicional `streq` y `strne` enviamos la orden al puerto que enciende los LEDs o al puerto que los apaga, respectivamente:

```
irq_handler:
    push    {r0, r1, r2}

    ldr     r0, =ledst      @ Leo puntero a v. ledst
    ldr     r1, [r0]        @ Leo variable
    eors    r1, #1          @ Invierto bit 0, act. flag Z
    str     r1, [r0]        @ Escribo variable

    ldr     r0, =GPBASE
/* guia bits      10987654321098765432109876543210 */
    ldr     r1, =0b00001000010000100000111000000000
    streq   r1, [r0, #GPSET0]
    strne   r1, [r0, #GPCLR0]
```

Luego escribimos un 1 en el M1 de `CS`, para resetear el estado de coincidencia, ya que de lo contrario el gestor de interrupciones verá el bit siempre a 1 y no lanzará más interrupciones. Resulta confuso tener que escribir un 1 en el puerto para almacenar un 0, pero de un modo similar a lo que ocurre con el puerto `GPCLR0` del GPIO es para ahorrar operaciones y que la RTI sea más rápida. Así no hay que leer el puerto, aplicar una máscara y volver a escribir en el mismo puerto, con una escritura es suficiente:

```
    ldr     r0, =STBASE
    mov     r1, #0b0010
    str     r1, [r0, #STCS]
```

Luego tenemos que actualizar el puerto comparador, de lo contrario tardará poco más de una hora en cambiar de estado el LED (es lo que tarda el contador en dar una vuelta completa). Para ello leemos el contador (`CL0`) y le añadimos 500000 al valor leído. Como cada cuenta equivale a un microsegundo, este añadido al contador supone medio segundo, lo que nos da la cadencia de un segundo que buscamos. El resultado de la suma lo escribimos en el comparador (`C1`):

```

ldr    r1, [r0, #STCL0]
ldr    r2, =500000      @1 Hz
add    r1, r2
str    r1, [r0, #STC1]

```

Por último restauramos los registros utilizados y salimos de la RTI. Más abajo tenemos la definición de la variable `ledst`, como no tenemos sección de datos aparte la ponemos al final del código:

```

pop    {r0, r1, r2}
subs   pc, lr, #4

ledst: .word    0

```

5.1.8. Control de LEDs rojos con pulsadores

En este ejemplo cambiamos de fuente de interrupción, en lugar del temporizador empleamos los pulsadores. Queremos que al pulsar un botón se encienda el LED rojo del mismo lado del pulsador, dejando el otro apagado.

El esquema sería el de la figura 5.8.

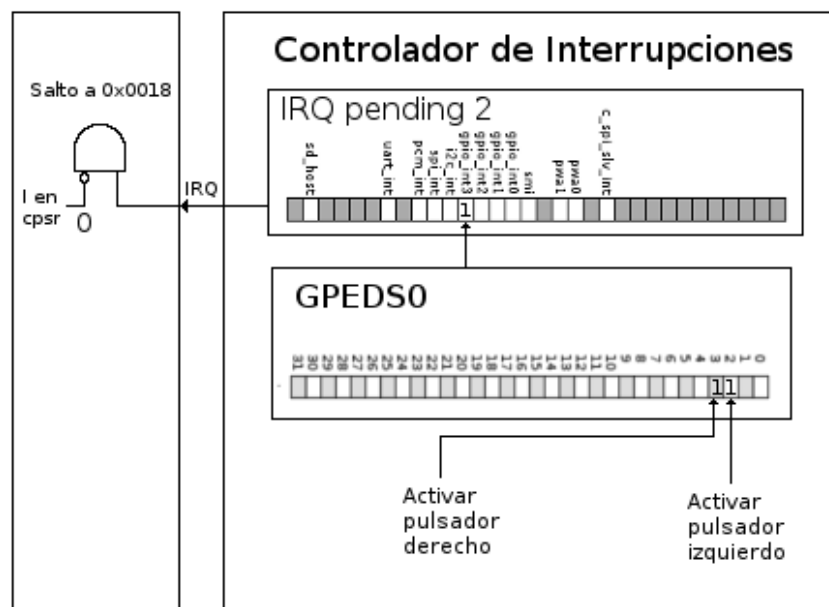


Figura 5.8: Interrupciones

Y el código fuente lo vemos a continuación:

Listado 5.4: inter3.s

```

        .include "inter.inc"
.text
/* Agrego vector interrupción */
        ADDEXC    0x18, irq_handler

/* Inicializo la pila en modos IRQ y SVC */
        mov     r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
        msr     cpsr_c, r0
        mov     sp, #0x8000
        mov     r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
        msr     cpsr_c, r0
        mov     sp, #0x8000000

/* Configuro GPIOs 9 y 10 como salida */
        ldr     r0, =GPBASE
        mov     r1, #0b00001000000000000000000000000000
        str     r1, [r0, #GPFSEL0]
/* guia bits          xx999888777666555444333222111000*/
        mov     r1, #0b00000000000000000000000000000001
        str     r1, [r0, #GPFSEL1]

/* Enciendo LEDs          10987654321098765432109876543210*/
        mov     r1, #0b0000000000000000000000001100000000
        str     r1, [r0, #GPSET0]

/* Habilito pines GPIO 2 y 3 (botones) para interrupciones*/
        mov     r1, #0b000000000000000000000000000001100
        str     r1, [r0, #GPFEN0]
        ldr     r0, =INTBASE

/* Habilito interrupciones, local y globalmente */
        mov     r1, #0b00000000000100000000000000000000
/* guia bits          10987654321098765432109876543210*/
        str     r1, [r0, #INTENIRQ2]
        mov     r0, #0b01010011    @ Modo SVC, IRQ activo
        msr     cpsr_c, r0

/* Repetir para siempre */
bucle:  b       bucle

```


5.1.9. Parpadeo secuencial de LEDs con sonido por altavoz

En este ejemplo vamos a trabajar con el temporizador, pero esta vez vamos a complicar un poco las cosas. En lugar de una fuente vamos a atender simultáneamente las peticiones de los comparadores C1 y C3.

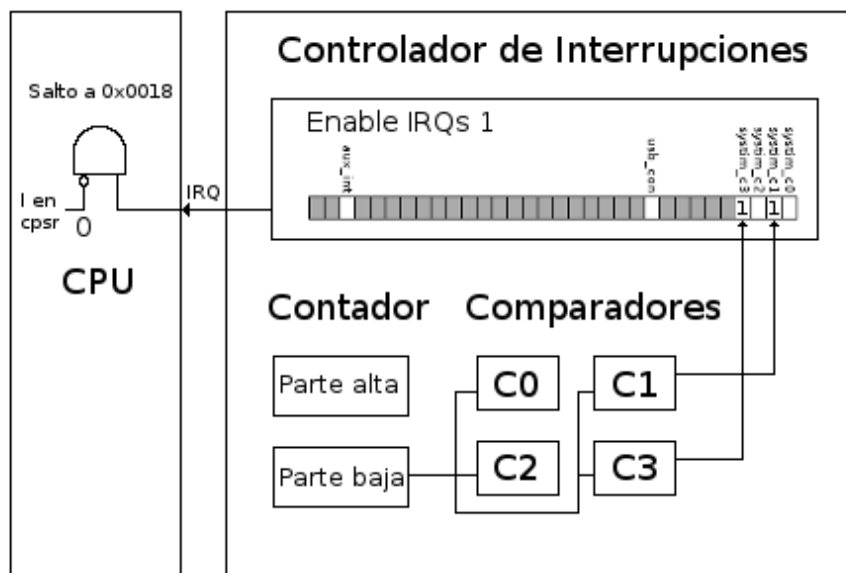


Figura 5.9: Interrupciones

Con esta segunda fuente vamos a controlar el altavoz, como podemos observar en la figura 5.9. Sacar un tono puro por el altavoz es equivalente a hacer parpadear un LED, lo único que cambia es que usamos otro pin distinto `GPIO_4` y aumentamos la frecuencia para que sea audible (a 1 Hz el oído humano no captaría sonido alguno). Utilizaremos la frecuencia estándar de afinación de 440 Hz, que coincide con el tono de espera de marcado en telefonía fija.

Por otro lado en lugar de hacer parpadear todos los LEDs lo que haremos es repetir una secuencia de 6 posiciones en la que en todo momento sólo uno de los 6 LEDs está encendido, que va cambiando de izquierda a derecha (aparentando movimiento) y cuando se llegue al sexto LED comenzamos de nuevo desde el primero. Para dar más sensación de movimiento disminuimos el periodo a 200 milisegundos.

La clave de todo está en saber cuál de los dos comparadores ha producido la interrupción (se puede dar el caso en que salten los dos a la vez). Ésto se puede hacer de dos formas distintas: o bien leemos el bit asociado `system_cx` en el puerto `IRQ_pending_1`, o bien leemos el `Mx` del puerto `CS`. Elegimos el segundo caso, así no gastamos otro puerto más para almacenar `INTBASE`.

El código completo del ejemplo es el siguiente:

Listado 5.5: inter4.s

```

        .include "inter.inc"
.text
/* Agrego vector interrupción */
        ADDEXC    0x18, irq_handler

/* Inicializo la pila en modos IRQ y SVC */
        mov     r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
        msr     cpsr_c, r0
        mov     sp, #0x8000
        mov     r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
        msr     cpsr_c, r0
        mov     sp, #0x8000000

/* Configuro GPIOs 4, 9, 10, 11, 17, 22 y 27 como salida */
        ldr     r0, =GPBASE
        ldr     r1, =0b00001000000000000000100000000000
        str     r1, [r0, #GPFSEL0]
/* guia bits          xx999888777666555444333222111000*/
        ldr     r1, =0b000000000010000000000000000001001
        str     r1, [r0, #GPFSEL1]
        ldr     r1, =0b0000000000100000000000000001000000
        str     r1, [r0, #GPFSEL2]

/* Programo C1 y C3 para dentro de 2 microsegundos */
        ldr     r0, =STBASE
        ldr     r1, [r0, #STCLO]
        add     r1, #2
        str     r1, [r0, #STC1]
        str     r1, [r0, #STC3]

/* Habilito interrupciones, local y globalmente */
        ldr     r0, =INTBASE
        mov     r1, #0b1010
        str     r1, [r0, #INTENIRQ1]
        mov     r0, #0b01010011    @ Modo SVC, IRQ activo
        msr     cpsr_c, r0

/* Repetir para siempre */
bucle:  b       bucle

```

```

/* Rutina de tratamiento de interrupción */
irq_handler:
    push    {r0, r1, r2, r3}

/* Leo origen de la interrupción */
    ldr     r0, =STBASE
    ldr     r1, =GPBASE
    ldr     r2, [r0, #STCS]
    ands    r2, #0b0010
    beq     sonido

/* Si es C1, ejecuto secuencia de LEDs */
    ldr     r2, =cuenta
/* guia bits      10987654321098765432109876543210 */
    ldr     r3, =0b00001000010000100000111000000000
    str     r3, [r1, #GPCLR0] @ Apago todos los LEDs
    ldr     r3, [r2]           @ Leo variable cuenta
    subs    r3, #1             @ Decremento
    moveq   r3, #6             @ Si es 0, volver a 6
    str     r3, [r2]           @ Escribo cuenta
    ldr     r3, [r2, +r3, LSL #2] @ Leo secuencia
    str     r3, [r1, #GPSET0] @ Escribo secuencia en LEDs

/* Reseteo estado interrupción de C1 */
    mov     r3, #0b0010
    str     r3, [r0, #STCS]

/* Programo siguiente interrupción en 200ms */
    ldr     r3, [r0, #STCL0]
    ldr     r2, =200000        @ 5 Hz
    add     r3, r2
    str     r3, [r0, #STC1]

/* ¿Hay interrupción pendiente en C3? */
    ldr     r3, [r0, #STCS]
    ands    r3, #0b0100
    beq     final              @ Si no, salgo

/* Si es C3, hago sonar el altavoz */
sonido: ldr     r2, =bitson
        ldr     r3, [r2]
        eors    r3, #1         @ Invierto estado
        str     r3, [r2]

```

```

        mov     r3, #0b10000      @ GPIO 4 (altavoz)
        streq   r3, [r1, #GPSET0] @ Escribo en altavoz
        strne   r3, [r1, #GPCLR0] @ Escribo en altavoz

/* Reseteo estado interrupción de C3 */
        mov     r3, #0b1000
        str     r3, [r0, #STCS]

/* Programo interrupción para sonido de 440 Hz */
        ldr     r3, [r0, #STCLO]
        ldr     r2, =1136          @ Contador para 440 Hz
        add     r3, r2
        str     r3, [r0, #STC3]

/* Recupero registros y salgo */
final:   pop     {r0, r1, r2, r3}
        subs    pc, lr, #4

bitson:  .word   0                @ Bit 0 = Estado del altavoz
cuenta:  .word   1                @ Entre 1 y 6, LED a encender
/* guia bits      7654321098765432109876543210 */
secuen:  .word   0b10000000000000000000000000000000
        .word   0b00000100000000000000000000000000
        .word   0b00000000001000000000000000000000
        .word   0b00000000000000000001000000000000
        .word   0b00000000000000000000100000000000
        .word   0b00000000000000000000100000000000

```

Como es muy parecido al ejemplo de antes, sólo vamos a comentar las diferencias que encontremos. La primera de ellas es que además de los 6 GPIOs de los LEDs, configuramos como salida un séptimo pin, el GPIO 4, para manejar el altavoz:

```

        ldr     r0, =GPBASE
        ldr     r1, =0b000010000000000000000001000000000000
        str     r1, [r0, #GPFSEL0]

```

El siguiente código es para incluir el comparador C3 (además del C1 que había anteriormente), tanto para proporcionar la primera interrupción como para habilitarla individualmente:

```

        ldr     r0, =STBASE
        ldr     r1, [r0, #STCLO]
        add     r1, #2
        str     r1, [r0, #STC1]
        str     r1, [r0, #STC3]

```

```

ldr    r0, =INTBASE
mov    r1, #0b1010
str    r1, [r0, #INTENIRQ1]

```

Ya hemos acabado con el programa principal, veamos ahora la RTI. Primero mostramos la estructura del código y luego las rutinas individuales tanto para el manejo de LEDs como para el altavoz:

```

irq_handler:
    push    {r0, r1, r2, r3}
    ldr     r0, =STBASE
    ldr     r1, =GPBASE
    ldr     r2, [r0, #STCS]
    ands    r2, #0b0010
    beq     sonido

    [ manejo de LEDs ]

    ldr     r3, [r0, #STCS]
    ands    r3, #0b0100
    beq     final
sonido:
    [ manejo de altavoz ]

final:    pop     {r0, r1, r2, r3}
          subs    pc, lr, #4

```

Los registros `r0` y `r1` los hacemos apuntar a la base del **System Timer** y del **GPIO** y no tocamos dichos valores durante toda la interrupción, vamos a estar constantemente leyendo y escribiendo puertos y resulta incómodo tener que cargar la base cada vez.

Es un error muy habitual suponer que la fuente de la interrupción sólo ha sido una, aunque la gran mayoría de las veces sea así se puede dar el caso de que coincidan los dos comparadores a la vez. De la misma forma si sabemos que sólo hay dos fuentes y una de ellas no ha provocado la interrupción, por descarte ha tenido que ser la otra, podemos ahorrarnos la comprobación.

El flujo sería el siguiente: leemos **M1** para ver si la interrupción la ha provocado el comparador de **C1**, si ha sido así ejecutamos el código de manejo de LEDs; si no, saltamos directamente al manejo del altavoz (sabemos seguro que la fuente viene de ahí).

Tras el código del manejo de LEDs leemos **M3** para saber si además de **C1** ha saltado también el comparador **C3**. Si no ha saltado, lo más normal, salimos por **final**; si lo ha hecho, procesamos la interrupción con el código de manejo del altavoz

para luego salir de la RTI.

Estos programas no son fáciles de crear y nunca funcionan a la primera. Es una buena práctica hacer funcionar por separado el código de los LEDs y el código del altavoz, y una vez comprobemos que funcionan, aglutinarlo en una única RTI. De esta forma aislamos lo máximo posible los errores que podamos cometer, es muy fácil equivocarse en una tontería y estar dándole vueltas al código sin encontrar el fallo. A diferencia de los primeros capítulos que disponíamos de `gdb`, en Bare Metal no tenemos acceso a ningún depurador.

Prosigamos ahora con el código de manejo de LEDs. Recordemos que hemos complicado un poco las cosas para emitir una secuencia en lugar de un simple parpadeo. Para ello mostramos el código seguido de las variables empleadas en el mismo:

```

        ldr        r2, =cuenta
/* guia bits      10987654321098765432109876543210*/
        ldr        r3, =0b00001000010000100000111000000000
        str        r3, [r1, #GPCLR0] @ Apago todos los LEDs
        ldr        r3, [r2]           @ Leo variable cuenta
        subs       r3, #1             @ Decremento
        moveq      r3, #6             @ Si es 0, volver a 6
        str        r3, [r2]           @ Escribo cuenta
        ldr        r3, [r2, +r3, LSL #2] @ Leo secuencia#2]
        str        r3, [r1, #GPSET0] @ Escribo secuencia en LEDs
        mov        r3, #0b0010
        str        r3, [r0, #STCS]
        ldr        r3, [r0, #STCLO]
        ldr        r2, =200000        @ 5 Hz
        add        r3, r2
        str        r3, [r0, #STC1]
        [...]
cuenta: .word      1                  @ Entre 1 y 6, LED a encender
/* guia bits      7654321098765432109876543210*/
secuen: .word      0b10000000000000000000000000000000
        .word      0b00000100000000000000000000000000
        .word      0b00000000001000000000000000000000
        .word      0b00000000000000000100000000000000
        .word      0b00000000000000000010000000000000
        .word      0b00000000000000000001000000000000

```

En la variable `cuenta` almacenamos un contador que va desde 6 hasta 1, que actúa como índice para el array `secuen`. Al decrementar aprovechamos la propia instrucción de resta para comprobar que se ha llegado al final de la cuenta (0), y en dicho caso restablecemos la cuenta a 6 mediante la instrucción de ejecución condicional `moveq`.

En el array `secuen` tenemos almacenadas las posiciones que corresponden a los LEDs dentro del puerto `GPSET0`, cada posición del array es para encender un LED en concreto. Antes de esto hemos apagado todos los LEDs enviando el valor que codifica todos los LEDs al puerto `GPCLR0`.

A parte de sacar la secuencia correspondiente debemos especificar cuándo será la siguiente interrupción. Como hicimos en el ejemplo anterior, esto se resuelve leyendo el valor del puerto `STCLO`, sumándole 200000 (200 milisegundos) y escribiéndolo en el comparador `STC1`.

Acabado el código de manejo de LEDs, ya sólo falta por explicar el manejo del altavoz:

```
sonido: ldr    r2, =bitson
        ldr    r3, [r2]
        eors   r3, #1           @ Invierto estado
        str    r3, [r2]
        mov    r3, #0b10000     @ GPIO 4 (altavoz)
        streq  r3, [r1, #GPSET0] @ Escribo en altavoz
        strne  r3, [r1, #GPCLR0] @ Escribo en altavoz
        mov    r3, #0b1000
        str    r3, [r0, #STCS]
        ldr    r3, [r0, #STCLO]
        ldr    r2, =1136        @ Contador para 440 Hz
        add    r3, r2
        str    r3, [r0, #STC3]
        [...]
bitson: .word 0
```

Es un calco de la rutina que hacía parpadear todos los LEDs, cambiando el valor que se envía a `GPCLR0/GPSET0`, el comparador que es `C3` en lugar de `C1`, y el valor que sumamos al temporizador, que se corresponde a 440 Hz en vez de a 1 Hz.

5.1.10. Manejo de FIQs y sonidos distintos para cada LED

Este ejemplo es muy parecido al anterior pero con cambios sutiles. El hecho de cambiar una de las dos IRQs por una FIQ incluso simplifica el código, ya que tienen distintas RTIs y en cada una la fuente de interrupción es única, por lo que no hay que comprobar nada ni hacer saltos.

Empecemos con el programa principal. Aquí sí que hay cambios porque tenemos que agregar un elemento nuevo al vector de interrupciones, inicializar el puntero de pila del modo FIQ y activar la fuente de interrupción FIQ local y globalmente:

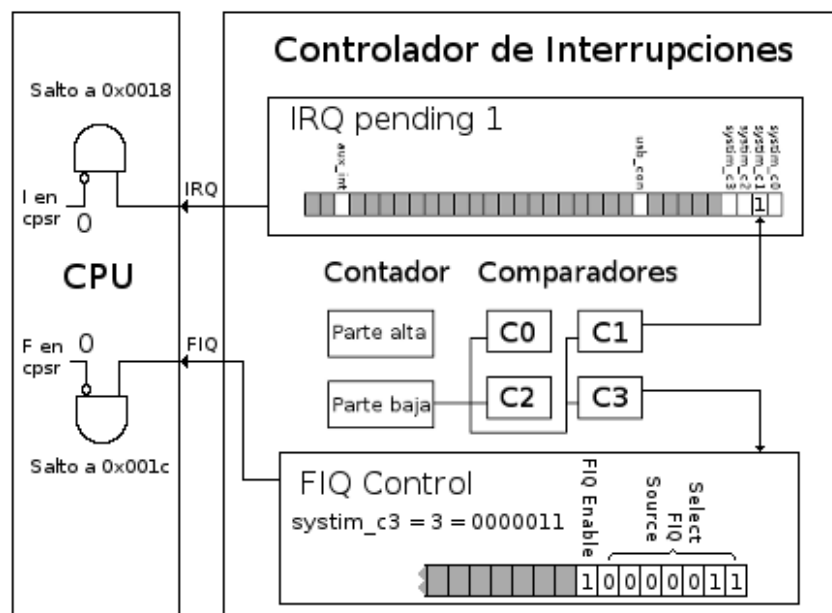


Figura 5.10: Interrupciones

Listado 5.6: Programa principal de inter5.s

```

/* Agrego vectores de interrupción */
ADDEXC 0x18, irq_handler
ADDEXC 0x1c, fiq_handler

/* Inicializo la pila en modos FIQ, IRQ y SVC */
mov     r0, #0b11010001    @ Modo FIQ, FIQ&IRQ desact
msr     cpsr_c, r0
mov     sp, #0x4000
mov     r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
msr     cpsr_c, r0
mov     sp, #0x8000
mov     r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
msr     cpsr_c, r0
mov     sp, #0x8000000

/* Configuro GPIOs 4, 9, 10, 11, 17, 22 y 27 como salida */
ldr     r0, =GPBASE
ldr     r1, =0b00001000000000000000000000000000
str     r1, [r0, #GPFSEL0]

/* guia bits          xx999888777666555444333222111000 */
ldr     r1, =0b0000000000010000000000000000000001001

```

```

        str    r1, [r0, #GPFSEL1]
        ldr    r1, =0b0000000000010000000000000001000000
        str    r1, [r0, #GPFSEL2]

/* Programa C1 y C3 para dentro de 2 microsegundos */
        ldr    r0, =STBASE
        ldr    r1, [r0, #STCLO]
        add    r1, #2
        str    r1, [r0, #STC1]
        str    r1, [r0, #STC3]

/* Habilito C1 para IRQ */
        ldr    r0, =INTBASE
        mov    r1, #0b0010
        str    r1, [r0, #INTENIRQ1]

/* Habilito C3 para FIQ */
        mov    r1, #0b10000011
        str    r1, [r0, #INTFIQCON]

/* Habilito interrupciones globalmente */
        mov    r0, #0b00010011    @ Modo SVC, FIQ&IRQ activo
        msr    cpsr_c, r0

/* Repetir para siempre */
bucle:  b      bucle

```

Queremos que FIQ se active con C3, que es el bit 3 del IRQ 1, por tanto índice 3 para la fuente FIQ. Como veis, la única pega que tienen las FIQs es que sólo admiten una fuente de interrupción. Además del índice ponemos el bit 7 a uno para indicar que queremos habilitar dicha fuente, siendo la constante 0b10000011.

Ahora veamos el manejador IRQ (la RTI) que, como hemos adelantado, es más sencilla que en el ejemplo anterior:

```

/* Rutina de tratamiento de interrupción IRQ */
irq_handler:
        push   {r0, r1, r2}
        ldr    r0, =GPBASE
        ldr    r1, =cuenta

/* Apago todos LEDs      10987654321098765432109876543210 */
        ldr    r2, =0b00001000010000100000111000000000
        str    r2, [r0, #GPCLR0]

```



```

        ldr    r2, [r1]           @ Leo variable cuenta
        subs   r2, #1            @ Decremento
        moveq  r2, #6            @ Si es 0, volver a 6
        str    r2, [r1], #-4     @ Escribo cuenta
        ldr    r2, [r1, +r2, LSL #3] @ Leo secuencia
        str    r2, [r0, #GPSET0] @ Escribo secuencia en LEDs

/* Reseteo estado interrupción de C1 */
        ldr    r0, =STBASE
        mov    r2, #0b0010
        str    r2, [r0, #STCS]

/* Programo siguiente interrupción en 500ms */
        ldr    r2, [r0, #STCLO]
        ldr    r1, =500000      @ 2 Hz
        add    r2, r1
        str    r2, [r0, #STC1]

/* Recupero registros y salgo */
        pop    {r0, r1, r2}
        subs   pc, lr, #4

```

Observamos que al acceder a la tabla `secuen` multiplicamos el índice por 8 en lugar de por 4. Esto es así porque hemos incluido en dicha tabla el valor de la longitud de onda (inverso de la frecuencia) con la que queremos que suene cada LED, la zona de datos es ésta:

```

bitson: .word 0           @ Bit 0 = Estado del altavoz
cuenta: .word 1           @ Entre 1 y 6, LED a encender
secuen: .word 0b10000000000000000000000000000000
        .word 716         @ Retardo para nota 6
        .word 0b00000100000000000000000000000000
        .word 758         @ Retardo para nota 5
/* guia bits 7654321098765432109876543210 */
        .word 0b00000000001000000000000000000000
        .word 851         @ Retardo para nota 4
        .word 0b00000000000000000000100000000000
        .word 956         @ Retardo para nota 3
/* guia bits 7654321098765432109876543210 */
        .word 0b00000000000000000000100000000000
        .word 1012        @ Retardo para nota 2
        .word 0b00000000000000000000010000000000
        .word 1136        @ Retardo para nota 1

```

Serían las notas puras que van después del LA estándar de 440 Hz (1136), cuyos semitonos se obtienen multiplicando la frecuencia por raíz duodécima de 2, que es aproximadamente 1,05946. Las notas serían, en hercios: LA (440), SI (493,88), DO (523,25), RE (587,33), MI (659,26) y FA (698,46).

Finalmente tenemos el manejador de FIQ asociado al altavoz. La elección de la fuente de interrupción no es arbitraria, hemos escogido FIQ para el altavoz porque se ejecutará más veces que el cambio de LEDs, concretamente 220 veces más con la nota más grave. En estos ejemplos no importa, pero en casos reales donde el tiempo de CPU es un recurso limitado, los ciclos que nos ahorramos con una FIQ en un proceso crítico pueden ser determinantes:

```
/* Rutina de tratamiento de interrupción FIQ */
fiq_handler:
    ldr    r8, =GPBASE
    ldr    r9, =bitson

/* Hago sonar altavoz invirtiendo estado de bitson */
    ldr    r10, [r9]
    eors   r10, #1
    str    r10, [r9], #4

/* Leo cuenta y luego elemento correspondiente en secuen */
    ldr    r10, [r9]
    ldr    r9, [r9, +r10, LSL #3]

/* Pongo estado altavoz según variable bitson */
    mov    r10, #0b10000    @ GPIO 4 (altavoz)
    streq   r10, [r8, #GPSET0]
    strne   r10, [r8, #GPCLR0]

/* Reseteo estado interrupción de C3 */
    ldr    r8, =STBASE
    mov    r10, #0b1000
    str    r10, [r8, #STCS]

/* Programo retardo según valor leído en array */
    ldr    r10, [r8, #STCLO]
    add    r10, r9
    str    r10, [r8, #STC3]

/* Salgo de la RTI */
    subs   pc, lr, #4
```

El código sería idéntico al de una IRQ si no fuera porque empleamos registros a partir de `r8` en lugar de a partir de `r0`, y no los salvaguardamos con las instrucciones `push/pop`. La razón es que el modo de operación FIQ es el único que tiene sus propios registros `r8-r12` (ver figura 5.2) con el objetivo de no perder el tiempo guardando y recuperando datos de la pila. En situaciones más críticas podemos incluso ubicar la RTI justo al final del vector de interrupciones. Esta tabla no contiene datos, sino instrucciones, y lo que hace la CPU cuando ocurre una excepción es saltar (ejecutar) a la dirección asociada en dicho vector. Así que cada elemento es una instrucción de salto que apunta a su RTI correspondiente, de no ser un salto se solaparía con el siguiente elemento del vector. Excepto el último elemento del vector, que no se solaparía con nada y que corresponde a las interrupciones FIQ. Se ha escogido intencionalmente así para ahorrarse el salto inicial.

5.1.11. Control de luces/sonido con pulsadores en lugar temporizadores

Los pulsadores izquierdo y derecho de nuestra placa externa están asociados a los puertos GPIO 2 y GPIO 3 respectivamente. Veremos cómo se genera una interrupción al pulsar cualquiera de los mismos.

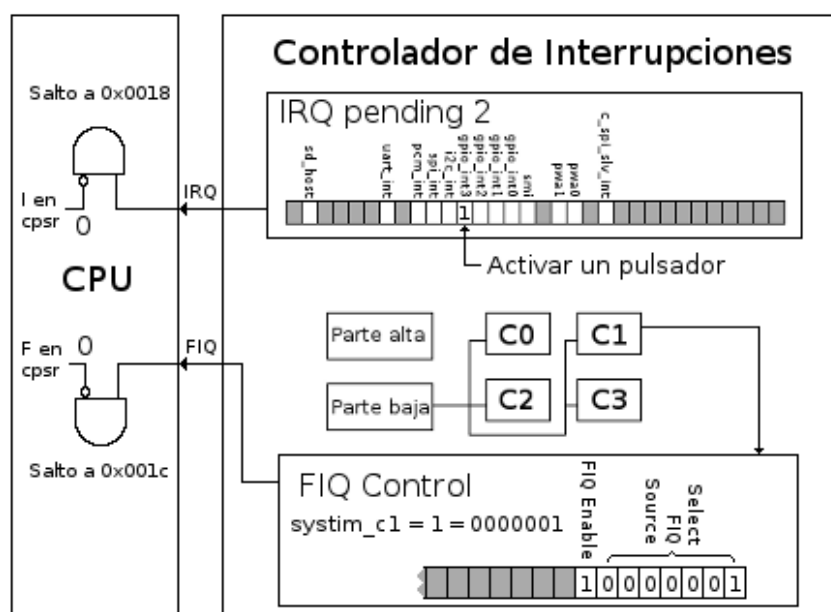


Figura 5.11: Interrupciones

El programa principal sería el siguiente:

Listado 5.7: Programa principal de inter6.s

```

/* Agrego vectores de interrupción */
    ADDEXC    0x18, irq_handler
    ADDEXC    0x1c, fiq_handler

/* Inicializo la pila en modos FIQ, IRQ y SVC */
    mov       r0, #0b11010001    @ Modo FIQ, FIQ&IRQ desact
    msr       cpsr_c, r0
    mov       sp, #0x4000
    mov       r0, #0b11010010    @ Modo IRQ, FIQ&IRQ desact
    msr       cpsr_c, r0
    mov       sp, #0x8000
    mov       r0, #0b11010011    @ Modo SVC, FIQ&IRQ desact
    msr       cpsr_c, r0
    mov       sp, #0x8000000

/* Configuro GPIOs 4, 9, 10, 11, 17, 22 y 27 como salida */
    ldr       r0, =GPBASE
    ldr       r1, =0b00001000000000000000100000000000
    str       r1, [r0, #GPFSEL0]
/* guia bits          xx999888777666555444333222111000*/
    ldr       r1, =0b000000000010000000000000000001001
    str       r1, [r0, #GPFSEL1]
    ldr       r1, =0b0000000000100000000000000001000000
    str       r1, [r0, #GPFSEL2]

/* Enciendo LEDs          10987654321098765432109876543210*/
    mov       r1, #0b0000000000000000000000000100000000
    str       r1, [r0, #GPSET0]

/* Habilito pines GPIO 2 y 3 (botones) para interrupciones*/
    mov       r1, #0b000000000000000000000000000001100
    str       r1, [r0, #GPFEN0]

/* Programo C1 para dentro de 2 microsegundos */
    ldr       r0, =STBASE
    ldr       r1, [r0, #STCLO]
    add       r1, #2
    str       r1, [r0, #STC1]

/* Habilito GPIO para IRQ */

```

```

        ldr        r0, =INTBASE
/* guia bits      10987654321098765432109876543210*/
        mov        r1, #0b00000000000010000000000000000000
        str        r1, [r0, #INTENIRQ2]

/* Habilito C1 para FIQ */
        mov        r1, #0b10000001
        str        r1, [r0, #INTFIQCON]

/* Habilito interrupciones globalmente */
        mov        r0, #0b00010011    @modo SVC, FIQ&IRQ activo
        msr        cpsr_c, r0

/* Repetir para siempre */
bucle:  b          bucle

```

Lo nuevo que vemos aquí es una escritura en el puerto GPFEN0. De esta forma le decimos al controlador de interrupciones que esos pines del GPIO serán los únicos que provoquen interrupciones, concretamente flancos con de bajada síncronos (justo en el momento en que el botón toca fondo).

El manejador FIQ es idéntico al del ejemplo anterior, saca el sonido que corresponde al LED por el altavoz, cambiando C3 por C1.

Lo más relevante de este ejemplo está en la RTI asociada a la IRQ, que es la siguiente:

```

irq_handler:
        push       {r0, r1, r2}
        ldr        r0, =GPBASE
        ldr        r1, =cuenta

/* Apago todos LEDs      10987654321098765432109876543210*/
        ldr        r2, =0b00001000010000100000111000000000
        str        r2, [r0, #GPCLR0]

/* Leo botón pulsado */
        ldr        r2, [r0, #GPEDS0]
        ands       r2, #0b00000000000000000000000000001000
        beq        incre

/* Si es botón izquierdo, decrementar */
        str        r2, [r0, #GPEDS0] @ Reseteo flag b. izq
        ldr        r2, [r1]          @ Leo variable cuenta
        subs       r2, #1             @ Decremento
        moveq      r2, #6             @ Si es 0, volver a 6

```

[illegible]

Tenemos una bifurcación (saltos condicionales) debido a que cada botón es una fuente distinta de interrupción y tenemos que distinguir qué botón se ha pulsado. Aquí por suerte tenemos un puerto totalmente análogo al **STCS** de los temporizadores. Se llama **GPEDS0** (también hay otro **GPEDS1** para los GPIOs de 32 a 53 que no necesitamos) y sirve tanto para saber qué fuente ha producido la interrupción como para resetear su estado (y así permitir volver a ser interrumpidos por el mismo pin GPIO).

Con la instrucción **ands** comprobamos si un determinado bit está a 1 y lo indicamos en el flag Z. También podría valer la instrucción **tst**, que tiene la ventaja de no destruir el registro a la salida (de la misma forma que **cmp** es el equivalente no destructivo de **subs**).

Y por último debemos sacar la secuencia inversa a la que teníamos para que al pulsar el botón izquierdo las luces vayan hacia la izquierda y que con el botón derecho vayan en el otro sentido. Si la secuencia de izquierda a derecha era (6, 5, 4, 3, 2, 1, 6, 5, 4...), la inversa sería (1, 2, 3, 4, 5, 6, 1...). Es decir, incrementamos y cuando llegamos a 7 lo convertimos en 1. Ésto se hace con el siguiente fragmento:

```
add    r2, #1           @ Incremento
cmp    r2, #7           @ Comparo si llego a 7
moveq  r2, #1           @ Si es 7, volver a 1
```

Nótese que aquí la opción destructiva `subs` (en lugar de `cmp`) no nos vale porque necesitamos el valor del registro después. Sí que podemos cambiarlo por un `teq` (la alternativa no destructiva de `eors`).

El ejemplo completo quedaría así.

Listado D.2: apend2.s

```

        .include  "inter.inc"
.text
        ldr      r0, =GPBASE
/* guia bits          xx999888777666555444333222111000*/
        mov      r1, #0b00000000000001000000000000000000
        str      r1, [r0, #GPFSEL1]
/* guia bits          xx999888777666555444333222111000*/
        mov      r1, #0b00000000001000000000000000000000
        str      r1, [r0, #GPFSEL4]
        mov      r1, #2
        str      r1, [r0, #GPPUD]
        bl       wait
/* guia bits          10987654321098765432109876543210*/
        mov      r1, #0b000000000000001000000000000000100
        str      r1, [r0, #GPPUDCLK0]
        bl       wait
        mov      r1, #0
        str      r1, [r0, #GPPUD]
        str      r1, [r0, #GPPUDCLK0]
/* guia bits          10987654321098765432109876543210*/
        mov      r2, #0b00000000000000010000000000000000
/* guia bits          32109876543210987654321098765432*/
        mov      r3, #0b00000000000000001000000000000000
bucle:   str      r2, [r0, #GPCLR0]    @ apago GPIO 16
        str      r3, [r0, #GPCLR1]    @ apago GPIO 47
        ldr      r1, [r0, #GPLEV0]
/* guia bits          10987654321098765432109876543210*/
        tst      r1, #0b000000000000001000000000000000100
        streq    r2, [r0, #GPSET0]    @ enciendo GPIO 16
        streq    r3, [r0, #GPSET1]    @ enciendo GPIO 47
        b        bucle

wait:    mov      r1, #50
wait1:   subs     r1, #1
        bne      wait1
        bx       lr

```

Comprobamos cómo ahora sí funciona, y mientras tenemos el botón presionado, el LED se enciende, apagándose en cuanto lo soltamos.